

Learn Home Automation with Arduino

Lesson 8: Integrating Voice Control

Integrating Voice Control - Study Notes

Key Concepts

1. **IFTTT (If This Then That)** – a cloud based service that links triggers (e.g., Google Assistant voice commands) to actions (e.g., webhook calls).
2. **Webhooks** – HTTP requests (GET/POST) that can be sent from IFTTT to an Arduino compatible device (e.g., ESP8266/ESP32) to invoke a specific URL.
3. **Arduino compatible Wi-Fi board** – typically an ESP8266 or ESP32, programmed with the Arduino IDE to listen for incoming webhook requests and control hardware (LEDs, relays, servos, etc.).
4. **Google Assistant integration** – using the “Google Assistant” service in IFTTT to capture spoken phrases and forward them to a webhook.
5. **Security basics** – using unique secret keys, HTTPS (or at least a random token), and optional IP filtering to prevent unauthorized control.

Summary

This lesson shows how to give an Arduino based project a voice controlled interface by routing Google Assistant commands through IFTTT to a webhook that the Arduino board can understand. First, you create an IFTTT applet: the **trigger** is a Google Assistant phrase (e.g., “Turn on the garden lights”), and the **action** is a **Webhooks** request that sends a GET or POST to a URL hosted on your ESP8266/ESP32.

On the hardware side, the board runs a simple web server (using the ESPAsyncWebServer or the built in Wi-Fi library). When the server receives the request, it parses the query string or JSON payload, extracts the command, and toggles the appropriate output pin. By keeping the payload small (e.g., `?state=on``) and protecting the endpoint with a secret token, you maintain a balance between ease of use and security.

The result is a seamless voice controlled system: you speak to any Google Assistant enabled device, IFTTT translates the phrase into an HTTP request, and the Arduino board reacts instantly, allowing you to control lights, fans, motors, or any other actuators without writing a custom mobile app.

Important Points

- **Create an IFTTT account** and enable the **Google Assistant** and **Webhooks** services.
- **Define a clear voice phrase** in the Google Assistant trigger (e.g., “Turn on the fan”).
- **Webhook URL format:** `http://<YOUR_ESP_IP>/control?token=YOUR_SECRET&state=on`` (GET) or a JSON POST body like `{ "token": "YOUR_SECRET", "state": "on" }`.
- **Arduino sketch basics:**

```
```cpp
```

```
#include <ESP8266WiFi.h>
```

```

#include <ESPAsyncWebServer.h>
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASS";
const char* secret = "YOUR_SECRET";
AsyncWebServer server(80);
void setup() {
 pinMode(D5, OUTPUT); // example output pin
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) delay(500);
 server.on("/control", HTTP_GET, [](AsyncWebServerRequest *request){
 if (!request->hasParam("token") || request->arg("token") != secret) {
 request->send(401, "text/plain", "Unauthorized");
 return;
 }
 String state = request->arg("state");
 digitalWrite(D5, state == "on" ? HIGH : LOW);
 request->send(200, "text/plain", "OK");
 });
 server.begin();
}
void loop() {}

```

- **Security tip:** Use a long, random token and never expose your Wi Fi credentials in the public repository.
- **Testing:** Use a browser or `curl` to hit the URL before linking to IFTTT to confirm the board responds correctly.
- **Debugging:** Enable Serial output (`Serial.begin(115200)`) and print incoming parameters to see what IFTTT actually sends.

---

## Examples

### Example 1 – Voice controlled LED

1. **Hardware:** ESP8266 (NodeMCU) + an LED on D5 (with resistor).
2. **IFTTT applet:**
  - **If:** Google Assistant !' "Turn **on** the desk light".
  - **Then:** Webhooks !' `GET` request to `http://192.168.1.45/control?token=abc123&state=on`.
  - Duplicate for "Turn **off** the desk light" with `state=off`.
3. **Result:** Saying the phrase to any Google Assistant device toggles the LED instantly.

### Example 2 – Controlling a Relay driven Pump

1. **Hardware:** ESP32 + 5 /V relay module connected to a water pump.
2. **Webhook payload (POST JSON):**

```

```json

```

```
{ "token":"pumpSecret", "action":"start", "duration":30 }  
...
```

3. **Arduino code snippet:**

```
```cpp  
server.on("/pump", HTTP_POST, [](AsyncWebServerRequest *request){
 NULL,
 [](AsyncWebServerRequest *request, uint8_t *data, size_t len, size_t index, size_t total) {
 DynamicJsonDocument doc(256);
 deserializeJson(doc, data);
 if (doc["token"] != "pumpSecret") { request->send(401); return; }
 if (String(doc["action"]) == "start") {
 int secs = doc["duration"] | 0;
 digitalWrite(RELAY_PIN, HIGH);
 delay(secs * 1000);
 digitalWrite(RELAY_PIN, LOW);
 }
 request->send(200, "application/json", "{\"status\":\"done\"}");
 });
}
```

### 4. **IFTTT voice command:** “Start the garden pump for 30 seconds”. The phrase triggers the POST request, the ESP32 runs the pump for the requested time, then stops automatically.

---

## Quick Review

1. What two IFTTT services are required to connect Google Assistant to an Arduino board?
2. How can you protect your webhook endpoint from unauthorized use?
3. Write the URL you would use for a GET request that turns a device **off**, assuming the secret token is `myToken``.
4. Which Arduino library is commonly used to create an asynchronous web server on ESP8266/ESP32?
5. In the pump example, how does the Arduino code know how long to run the pump?

---

## Further Reading

- **MQTT vs. Webhooks** – when to use a lightweight publish/subscribe model for IoT control.
- **HTTPS on ESP8266/ESP32** – using `ESP8266WebServerSecure`` or `WiFiClientSecure`` for encrypted webhook calls.
- **Google Assistant “Routines”** – combining multiple actions (e.g., turn on lights *and* start music).
- **Node RED integration** – visual flow based programming for more complex voice triggered automations.
- **Security best practices for IoT devices** – token rotation, firmware OTA updates, and network segmentation.

